Practical 3: Basics of Quantum Circuits and Operations

In [3]:
```
from qiskit import QuantumCircuit
QuantumCircuit(2)
```

Out[3]:
```
<qiskit.circuit.quantumcircuit.QuantumCircuit at 0x222880c1190>
```

In [4]:
```
QuantumCircuit(2, 2)
```

Out[4]:
```
<qiskit.circuit.quantumcircuit.QuantumCircuit at 0x2228c0225d0>
```

In [5]:
```
import math

qc = QuantumCircuit(4)
qc.h(0)
qc.id(2)

qc.p(math.pi/2,0)
qc.rx(math.pi/4,2)
qc.ry(math.pi/8,0)
qc.rz(math.pi/2,1)
qc.s(3)
qc.sdg(3)
qc.sx(2)
qc.t(1)
qc.tdg(1)
qc.u(math.pi/2,0,math.pi,1)
qc.x(3)
qc.y([0,2,3])
qc.z(2)
```

Out[5]:
```
<qiskit.circuit.instructionset.InstructionSet at 0x2228c323010>
```

In [6]:
```
qc = QuantumCircuit(4)
qc.h(0)
qc.id(2)
qc.p(math.pi/2,0)
qc.rx(math.pi/4,2)
qc.ry(math.pi/8,0)
qc.rz(math.pi/2,1)
qc.s(3)
qc.sdg(3)
qc.sx(2)
qc.t(1)
qc.tdg(1)
qc.u(math.pi/2,0,math.pi,1)
qc.x(3)
qc.y([0,2,3])
```

```
qc.z(2)
qc.draw('latex')
```

Out[6]:

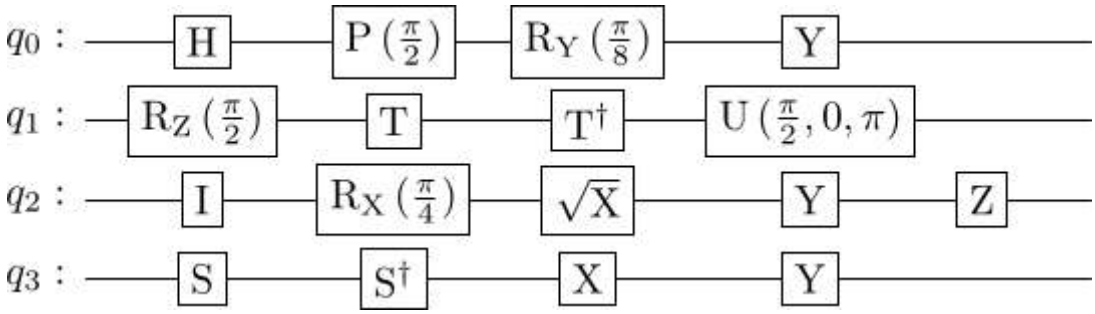


Table 1-2 - Page 16-18

In [8]:

```
qc.ccx(0,1,2)
qc.ch(0,1)
qc.cp(math.pi/4,0,1)
qc.crx(math.pi/2,2,3)
qc.crz(math.pi/4,0,1)
qc.cswap(0,2,3)

qc.csx(0,1)
qc.cu(math.pi/2,0,math.pi,0,0,1)
qc.cx(2,3)

qc.cy(2,3)
qc.cz(1,2)
qc.dcx(2,3)
qc.iswap(0,1)
qc.mcp(math.pi/4,[0,1,2],3)
qc.mcx([0,1,2],3)
qc.swap(2,3)
```

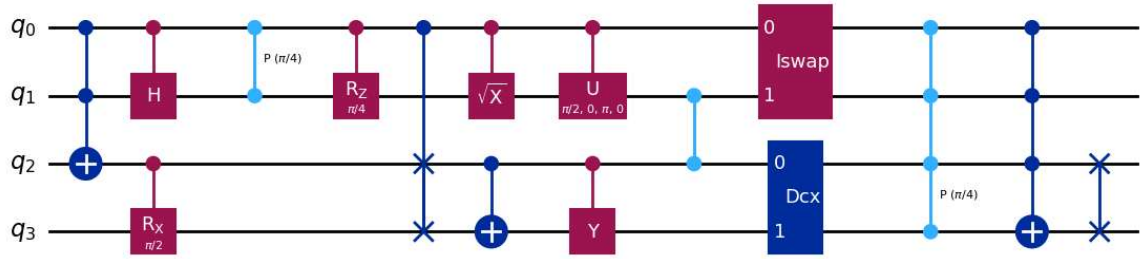
Out[8]:

```
<qiskit.circuit.instructionset.InstructionSet at 0x22289deb0a0>
```

In [9]:

```
qc = QuantumCircuit(4)
qc.ccx(0,1,2)
qc.ch(0,1)
qc.cp(math.pi/4,0,1)
qc.crx(math.pi/2,2,3)
qc.crz(math.pi/4,0,1)
qc.cswap(0,2,3)
qc.csx(0,1)
qc.cu(math.pi/2,0,math.pi,0,0,1)
qc.cx(2,3)
qc.cy(2,3)
qc.cz(1,2)
qc.dcx(2,3)
qc.iswap(0,1)
qc.mcp(math.pi/4,[0,1,2],3)
qc.mcx([0,1,2],3)
qc.swap(2,3)
qc.draw('mpl')
```

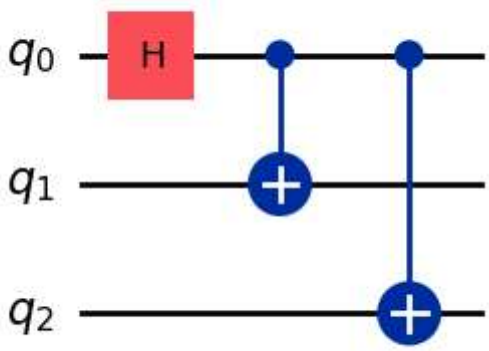
Out[9]:



In [10]:

```
qc = QuantumCircuit(3)
qc.h(0)
qc.cx(0, 1)
qc.cx(0, 2)
qc.draw('mpl')
```

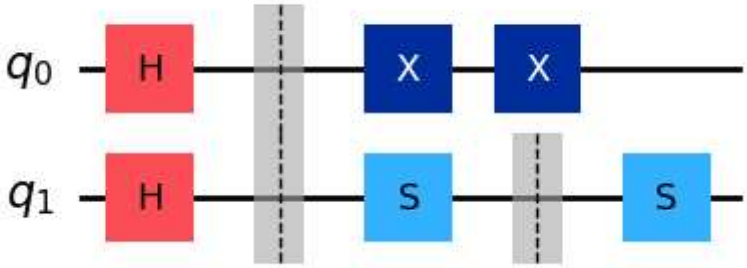
Out[10]:



In [11]:

```
qc = QuantumCircuit(2)
qc.h([0,1])
qc.barrier()
qc.x(0)
qc.x(0)
qc.s(1)
qc.barrier([1])
qc.s(1)
qc.draw('mpl')
```

Out[11]:

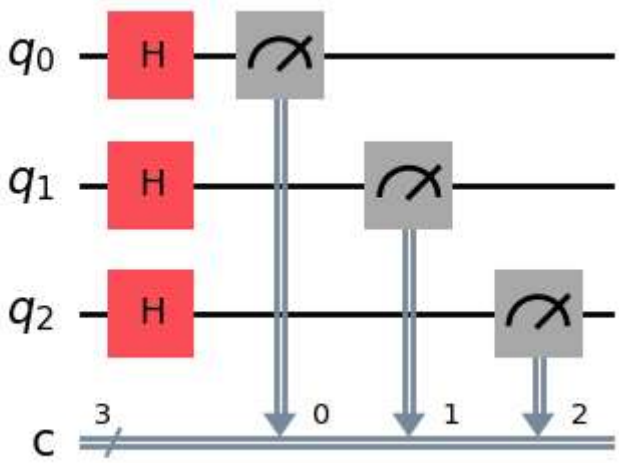


In [12]:

```
qc = QuantumCircuit(3, 3)
qc.h([0,1,2])
```

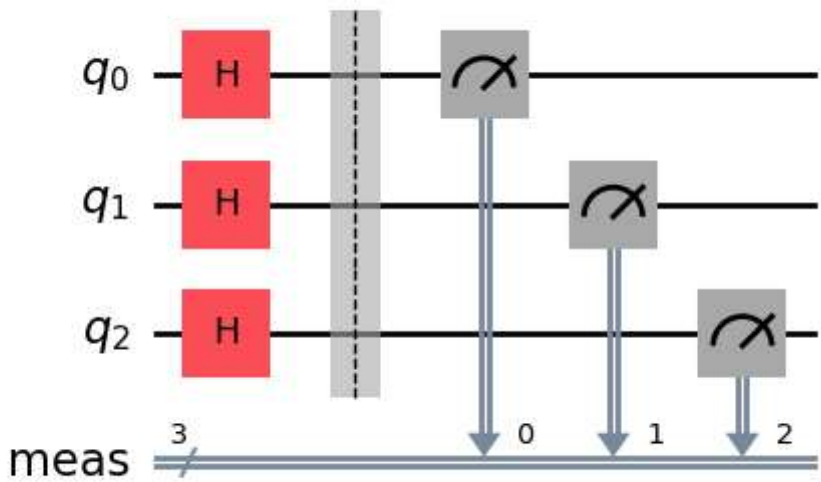
```
qc.measure([0,1,2], [0,1,2])
qc.draw('mpl')
```

Out[12]:



In [13]:
```
qc = QuantumCircuit(3)
qc.h([0,1,2])
qc.measure_all()
qc.draw('mpl')
```

Out[13]:



In [14]:
```
qc.depth()
```

Out[14]: 2

In [15]:
```
qc.size()
```

Out[15]: 6

In [16]:
```
qc.width()
```

Out[16]: 6

In [17]:
```
qc.clbits
```

Out[17]:
```
[Clbit(ClassicalRegister(3, 'meas'), 0),
 Clbit(ClassicalRegister(3, 'meas'), 1),
 Clbit(ClassicalRegister(3, 'meas'), 2)]
```

In [18]:
```
qc.data
```

Out[18]:
```
[CircuitInstruction(operation=Instruction(name='h', num_qubits=1, num_clbits=0, pa
rams=[]), qubits=(Qubit(QuantumRegister(3, 'q'), 0),), clbits=()), CircuitInstruct
ion(operation=Instruction(name='h', num_qubits=1, num_clbits=0, params=[]), qubits
=(Qubit(QuantumRegister(3, 'q'), 1),), clbits=()), CircuitInstruction(operation=In
struction(name='h', num_qubits=1, num_clbits=0, params=[]), qubits=(Qubit(QuantumR
egister(3, 'q'), 2),), clbits=()), CircuitInstruction(operation=Instruction(name
='barrier', num_qubits=3, num_clbits=0, params=[]), qubits=(Qubit(QuantumRegister
(3, 'q'), 0), Qubit(QuantumRegister(3, 'q'), 1), Qubit(QuantumRegister(3, 'q'),
2)), clbits=()), CircuitInstruction(operation=Instruction(name='measure', num_qubi
ts=1, num_clbits=1, params=[]), qubits=(Qubit(QuantumRegister(3, 'q'), 0),), clbit
s=(Clbit(ClassicalRegister(3, 'meas'), 0),)), CircuitInstruction(operation=Instruc
tion(name='measure', num_qubits=1, num_clbits=1, params=[]), qubits=(Qubit(Quantum
Register(3, 'q'), 1),), clbits=(Clbit(ClassicalRegister(3, 'meas'), 1),)), Circuit
Instruction(operation=Instruction(name='measure', num_qubits=1, num_clbits=1, para
ms=[]), qubits=(Qubit(QuantumRegister(3, 'q'), 2),), clbits=(Clbit(ClassicalRegist
er(3, 'meas'), 2),))]
```

In [19]:
```
qc.global_phase
```

Out[19]: 0.0

In [20]:
```
qc.num_clbits
```

Out[20]: 3

In [21]:
```
qc.num_qubits
```

Out[21]: 3

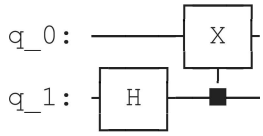
In [22]:
```
qc.qubits
```

Out[22]:
```
[Qubit(QuantumRegister(3, 'q'), 0),
 Qubit(QuantumRegister(3, 'q'), 1),
 Qubit(QuantumRegister(3, 'q'), 2)]
```

In [23]:
```
from qiskit.circuit.library import CXGate

qc = QuantumCircuit(2)
qc.h(1)
cx_gate = CXGate()
qc.append(cx_gate, [1,0])
qc.draw()
```

Out[23]:



In [24]:

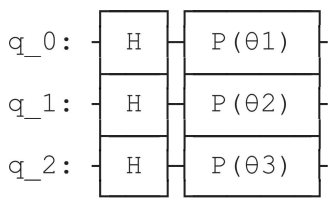
```
from qiskit.circuit import QuantumCircuit,\
                           Parameter

theta1 = Parameter('θ1')
theta2 = Parameter('θ2')
theta3 = Parameter('θ3')

qc = QuantumCircuit(3)
qc.h([0,1,2])
qc.p(theta1,0)
qc.p(theta2,1)
qc.p(theta3,2)

qc.draw()
```

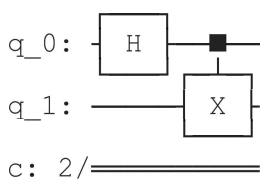
Out[24]:



In [25]:

```
qc = QuantumCircuit(2,2)
qc.h(0)
another_qc = QuantumCircuit(2,2)
another_qc.cx(0,1)
bell_qc = qc.compose(another_qc)
bell_qc.draw()
```

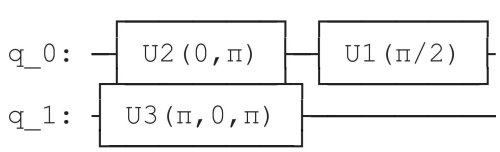
Out[25]:



In [26]:

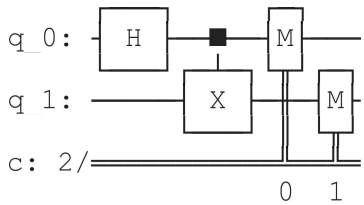
```
qc = QuantumCircuit(2)
qc.h(0)
qc.s(0)
qc.x(1)
decomposed_qc = qc.decompose()
decomposed_qc.draw()
```

Out[26]:



In [27]:
```
qasm_str = """
OPENQASM 2.0;
include "qelib1.inc";
qreg q[2];
creg c[2];
h q[0];
cx q[0],q[1];
measure q[0] -> c[0];
measure q[1] -> c[1];
"""
new_qc = QuantumCircuit.from_qasm_str(qasm_str)
new_qc.draw()
```

Out[27]:

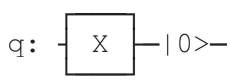


In [28]:
```
qc = QuantumCircuit(2)
qc.initialize([0, 0, 0, 1])
qc.draw()
```

Out[28]:

In [29]:
```
qc = QuantumCircuit(1)
qc.x(0)
qc.reset(0)
qc.draw()
```

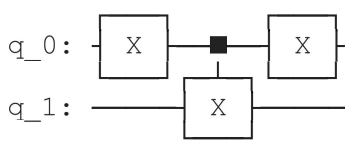
Out[29]:



In [30]:
```
anti_cnot_qc = QuantumCircuit(2)

anti_cnot_qc.x(0)
anti_cnot_qc.cx(0,1)
anti_cnot_qc.x(0)
anti_cnot_qc.draw()
```

Out[30]:



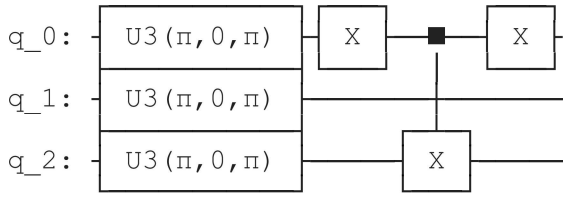
In [31]:
```
anti_cnot_gate = anti_cnot_qc.to_gate()

qc = QuantumCircuit(3)
qc.x([0,1,2])
```

```
qc.append(anti_cnot_gate, [0,2])

qc.decompose().draw()
```

Out[31]:

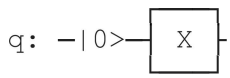


In [32]:

```
reset_one_qc = QuantumCircuit(1)

reset_one_qc.reset(0)
reset_one_qc.x(0)
reset_one_qc.draw()
```

Out[32]:



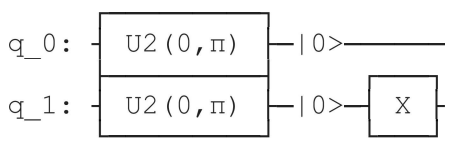
In [33]:

```
reset_one_inst = reset_one_qc.to_instruction()

qc = QuantumCircuit(2)
qc.h([0,1])
qc.reset(0)
qc.append(reset_one_inst, [1])

qc.decompose().draw()
```

Out[33]:



In [34]:

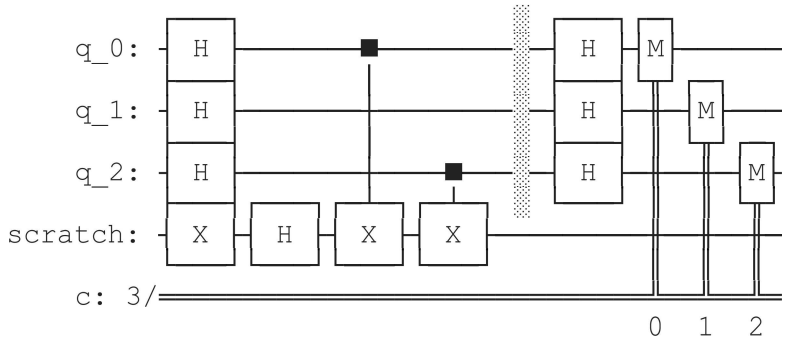
```
from qiskit import QuantumRegister, \
                   ClassicalRegister

qr = QuantumRegister(3, 'q')
scratch = QuantumRegister(1, 'scratch')
cr = ClassicalRegister(3, 'c')
qc = QuantumCircuit(qr, scratch, cr)

qc.h(qr)
qc.x(scratch)
qc.h(scratch)
qc.cx(qr[0], scratch)
qc.cx(qr[2], scratch)
qc.barrier(qr)
qc.h(qr)
qc.measure(qr, cr)

qc.draw()
```

Out[34]:



In [35]:

```
#Write a Qiskit script that creates |Φ+⟩, measures both qubits, runs 1024 shots on A

from qiskit_aer import AerSimulator
from qiskit import transpile

qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)

qc.measure([0, 1], [0, 1])

sim = AerSimulator()
compiled = transpile(qc, sim)
result = sim.run(compiled, shots=1024).result()

counts = result.get_counts()
print(counts)
```

```
{'00': 498, '11': 526}
```